

SHIPGOTCHI BUILD OPERATING MANUAL

Prompt Runbook

A paste-by-paste sequence for taking ShipGotchi from empty workspace to production submission, including specialist prompts for implementation, sprite generation, QA, deployment, and judging.

14 Prompts

Parallel Agents

Image Prompts

QA + Deploy

Submission Ready

ShipGotchi Prompt Runbook

Date: 2026-05-22 Purpose: paste-by-paste prompts for building ShipGotchi from an empty project to a submitted build.

How To Use This Runbook

Use the prompts in order. Each prompt is written so it can be pasted into Codex or a specialist agent with minimal extra context. When running multiple agents, give each agent only the relevant prompt plus the planning docs it needs.

Recommended order:

1. Master setup prompt
2. Architecture and scaffold prompt
3. Core data/scoring agent prompt
4. Frontend experience agent prompt
5. Sprite/art direction agent prompt
6. Builder DNA/copy agent prompt
7. Integration prompt
8. QA and hardening prompt
9. Deployment prompt
10. Submission prompt

Do not add new features unless the five MVP features are stable.

Prompt 1: Master Setup

We are building ShipGotchi for the Dollar Vibe Club 24-Hour Open Build Sprint, Track 2: Vibe Gotchi.

Read the planning docs in this workspace before making implementation decisions:

- docs/00-shipgotchi-planning-index.md
- docs/01-product-brief.md
- docs/02-mvp-specification.md
- docs/03-experience-visual-direction.md
- docs/04-technical-plan.md
- docs/05-execution-plan-agent-workstreams.md
- docs/06-judging-demo-submission-kit.md
- docs/07-prompt-runbook.md

Locked decisions (see planning index): solo full-stack builder, Next.js + TypeScript + Tailwind, Vercel hosting, a REQUIRED /api/github serverless proxy, and a layered (one-base) sprite system.

Goal:

Build a no-login, mobile-first consumer webapp where a visitor enters any public GitHub username and instantly hatches a premium digital pet whose mood, evolution, outfit, and Builder DNA are based on public GitHub activity.

MVP features:

1. GitHub Hatch
2. Pet Mood and Evolution
3. Premium Sprite Moment
4. Builder DNA Card
5. Compare / Party Mode

Rules:

- No login required.
- Webapp only.
- Public production URL.
- Mobile-friendly.
- Minimum 3 interactive features.
- Keep scope tight.
- Prioritize a stable judging demo over extra features.

First task:

Inspect the workspace, identify whether a project already exists, and recommend the fastest implementation path within the locked stack (Next.js + TS + Tailwind on Vercel). If empty, scaffold it. Do not implement features yet until you summarize the scaffold choice and file structure.

Prompt 2: Scaffold And Architecture

Implement the ShipGotchi app scaffold.

Stack is locked: Next.js (App Router) + TypeScript + Tailwind CSS. We use Next specifically so the GitHub proxy lives in the same project as the app/api routes and deploys to Vercel with zero extra config. Do not waste time on a complex design-system setup.

Create a clean architecture:

- app shell / main page
- app/api/github route (serverless proxy – see Prompt 3)
- components for hatch form, loading ritual, pet display, Builder DNA card, compare mode, and error states
- a client service that fetches from /api/github (NOT the GitHub API directly)
- pure utility module for scoring and pet profile generation
- sample data fixtures for demo fallback
- asset folder for pet sprites

Constraints:

- No auth, no login.
- No database required.
- No runtime AI image generation.
- The UI must work with sample data even if the proxy/GitHub fails.
- Mobile-first.

After scaffolding:

1. Run install/build/lint and report results.
2. Deploy the empty scaffold to Vercel so a live production URL exists immediately, then report the URL.

Prompt 3: Core Data And Scoring Agent

You are the Core Data and Scoring agent for ShipGotchi.

Read:

- docs/02-mvp-specification.md
- docs/04-technical-plan.md
- docs/05-execution-plan-agent-workstreams.md

Your task:

Implement the GitHub proxy, data layer, and deterministic pet scoring system.

Deliver:

0. A serverless proxy at app/api/github (GET ?username=) that:

- reads GITHUB_TOKEN from env (server-side only, never sent to the client)
- fetches profile + recent public events + recent repos server-side
- caches responses ~5 minutes to conserve GitHub quota
- returns one normalized payload, or a typed error mapping 404 → notFound, 403/rate-limit →

rateLimited, 5xx/network → networkError

Why: unauthenticated GitHub is 60 req/hr per IP and will die during live judging; an authenticated proxy gives 5,000/hr plus caching.

1. A client GitHub service that fetches from /api/github (NOT api.github.com directly).

2. Normalization logic that produces a GitHubActivity object.

3. A createPetProfile(activity) function that returns:

- username
- avatarUrl
- githubUrl
- level
- mood
- evolutionStage
- builderClass
- topLanguage
- energyScore
- activitySummary
- personalityLine
- spriteVariant
- outfitTrait

4. Sample fixture profiles for:

- very active builder
- normal builder
- sleepy/inactive builder
- old/legend account
- invalid/rate-limited fallback

5. Unit tests or simple verification scripts for scoring outputs if the project has a test setup.

Important:

- Keep scoring simple, explainable, and deterministic.
- Never make low activity feel insulting.
- Handle rate limits, not found, network failure, and sparse data.
- Return a UI-friendly error state instead of throwing raw errors into components.

Acceptance:

Different sample users should produce visibly different moods, levels, classes, and sprite variants.

Prompt 4: Frontend Experience Agent

You are the Frontend Experience agent for ShipGotchi.

Read:

- docs/01-product-brief.md
- docs/02-mvp-specification.md
- docs/03-experience-visual-direction.md
- docs/05-execution-plan-agent-workstreams.md

Your task:

Build the complete user flow from first screen to result.

Screens/states:

1. Landing / hatch state
2. Loading ritual
3. Result state
4. Error state
5. Compare / Party Mode state

Required UI:

- Brand: ShipGotchi
- Tagline: "Paste your GitHub. Meet the creature your commits created."
- Username input
- Hatch button
- Sample username chips
- Loading ritual with playful scan steps
- Pet result section
- Builder DNA card section
- Compare another builder action
- Up to 4 local comparison cards

Design direction:

- Cute premium toy.
- Light RPG stats.
- Soft builder-lab accents.
- Mobile-first.
- No dense dashboard feeling.

Important:

- The pet is the hero, not a small decoration.
- Keep text concise.
- Make the first successful hatch feel delightful within 10 seconds.
- Avoid feature explanation blocks on the page.

Acceptance:

A judge can open the site, click a sample username, see a pet result, and compare another username without needing instructions.

Prompt 5: Sprite And Art Direction Agent

You are the Sprite and Art Direction agent for ShipGotchi.

Read:

- docs/03-experience-visual-direction.md
- docs/02-mvp-specification.md

Your task:

Build the visual pet system as ONE base mascot with state-driven layers – not 7 independent images. Layers: body tint, expression, accessory, and motion. This guarantees the variants read as the same creature and avoids fighting an image model for consistency.

Art direction:

Cute premium toy, soft digital collectible, expressive, friendly, mobile-friendly, screenshot-worthy. Add only light builder-lab/RPG flavor.

State targets (driven by the layers, all the same creature):

1. Egg
2. Sprout
3. Builder
4. Evolved
5. Overclocked
6. Sleepy
7. Mystery/error fallback

Optional overlays (accessory layer, by top language):

- TypeScript/JavaScript: star visor
- Python: soft wizard cap
- Rust: bronze armor accent
- Go: runner goggles
- CSS/HTML: paint scarf

Primary deliverable:

- An SVG/CSS sprite component that takes (spriteVariant, mood, outfitTrait) and renders the right layered combination, with light CSS/Framer Motion animation (idle bob, hatch pop, sleepy z's).

Stretch deliverable (only after the 5 MVP features are stable):

- Swap in pre-generated AI renders per variant using runbook Prompt 6. Never make the demo depend on these.

Constraints:

- Do not depend on runtime AI image generation.
- Transparent or clean backgrounds.
- Variants must stay visually consistent as the same creature.
- Pet must be readable at mobile sizes.

Acceptance:

Each pet state must look distinct, premium, and emotionally readable.

Prompt 6: AI Image Generation Prompts

Use these prompts with an image-generation service if we create sprite assets externally.

Global Style Prompt

Create a cute premium digital toy mascot for a webapp called ShipGotchi. The mascot is a small builder companion born from GitHub commits. Style: soft 3D vinyl toy, collectible, expressive, friendly, polished, bright, high-quality product render, clean silhouette, mobile app mascot, subtle developer-lab details, not dark, not cyberpunk-heavy, not anime-heavy. Transparent background. Same character identity across variants.

Egg Variant

Create the Egg form of the ShipGotchi mascot. A cute premium toy egg with tiny glowing code-symbol freckles, soft rounded shape, curious expression hinted through a small crack of light, warm and friendly. Soft 3D vinyl collectible style, transparent background, clean silhouette, mobile app asset.

Sprout Variant

Create the Sprout form of the ShipGotchi mascot. Same character as the egg, now a tiny newly hatched builder creature with small rounded body, bright eyes, tiny leaf-like antenna shaped like code brackets, holding a small commit spark. Cute premium toy, soft 3D vinyl collectible, transparent background.

Builder Variant

Create the Builder form of the ShipGotchi mascot. Same character, now confident and active, wearing tiny soft builder goggles and a small utility scarf, holding a glowing commit cube. Cute premium toy, soft 3D vinyl collectible, expressive, friendly, transparent background.

Evolved Variant

Create the Evolved form of the ShipGotchi mascot. Same character, more polished and heroic, larger expressive eyes, subtle badge on chest, tiny floating code sparks around it, premium collectible toy, playful RPG evolution energy, bright and friendly, transparent background.

Overclocked Variant

Create the Overclocked form of the ShipGotchi mascot. Same character, excited and powered up, tiny glowing goggles, energetic pose, harmless bright commit sparks, premium cute toy, celebratory not aggressive, soft 3D vinyl collectible, transparent background.

Sleepy Variant

Create the Sleepy form of the ShipGotchi mascot. Same character, cozy and napping, soft blanket or tiny hoodie, dim commit spark beside it, sweet and lovable, not sad or punished, premium toy style, transparent background.

Mystery Variant

Create the Mystery fallback form of the ShipGotchi mascot. Same character partly hidden in a soft translucent mystery bubble, curious eyes visible, gentle question mark-shaped glow, friendly error state, premium cute toy style, transparent background.

Prompt 7: Builder DNA And Copy Agent

You are the Builder DNA and Copy agent for ShipGotchi.

Read:

- docs/02-mvp-specification.md
- docs/03-experience-visual-direction.md
- docs/06-judging-demo-submission-kit.md

Your task:

Create the personality/copy system that makes the app feel alive.

Deliver:

1. Builder class names mapped to common activity/language patterns.
2. Mood labels and descriptions.
3. Evolution descriptions.
4. Loading ritual copy.
5. Personality lines for each mood/class combination.
6. Error copy for invalid username, rate limit, network failure, and sparse data.
7. Final submission description under 500 characters.

Tone:

Warm, playful, premium, builder-native, never cruel.

Avoid:

- Long explanations
- Mean roasts
- Overused developer memes
- Copy that requires insider knowledge to understand

Acceptance:

The copy should make the result feel personal and screenshot-worthy while staying clear for judges.

Prompt 8: Integration Prompt

Integrate the ShipGotchi workstreams into one polished product.

Read all docs in /docs, especially:

- docs/02-mvp-specification.md
- docs/03-experience-visual-direction.md
- docs/04-technical-plan.md
- docs/05-execution-plan-agent-workstreams.md

Tasks:

1. Wire the hatch form to the GitHub data service.
2. Convert GitHub data into a PetProfile.
3. Render the correct pet sprite variant from PetProfile.spriteVariant.
4. Render Builder DNA from the PetProfile contract.
5. Add local Compare Mode for up to 4 users.
6. Add sample username chips and bundled sample data fallback.
7. Ensure loading, success, not found, rate-limited, and network error states all look intentional.
8. Make the app responsive and polished on mobile.

Do not add new major features.

Acceptance:

The full user flow works from a clean page load:

landing → hatch → loading → pet result → compare mode → share/result card.

Prompt 9: QA And Hardening Agent

You are the QA and Hardening agent for ShipGotchi.

Read:

- docs/02-mvp-specification.md
- docs/04-technical-plan.md
- docs/06-judging-demo-submission-kit.md

Your task:

Break the app before judges do.

Test:

1. Valid active GitHub usernames.
2. Valid inactive GitHub usernames.
3. Invalid usernames.
4. Empty input.
5. GitHub API rate-limit behavior.
6. Network failure if possible.
7. Mobile viewport.
8. Desktop viewport.
9. Compare Mode with 2, 3, and 4 users.
10. Fresh production deployment URL.

Required fixes:

- No raw errors visible to users.
- No login required.
- No layout overlap on mobile.
- No broken sprite states.
- Sample data path must work without GitHub.

Report:

- What passed.
- What failed.
- What was fixed.
- Remaining risks.

Prompt 10: Deployment Agent

Deploy ShipGotchi to a public production URL.

Read:

- docs/06-judging-demo-submission-kit.md

Tasks:

1. Build the app locally.
2. Fix build errors.
3. Deploy to Vercel (Git-connected, auto-deploy on push).
4. Confirm GITHUB_TOKEN is set in Vercel env and the /api/github proxy returns live data in production.
5. Verify the production URL in a browser.
6. Test the core judge path on production:
 - open URL
 - click sample username
 - hatch result
 - compare another user
 - view Builder DNA
7. Confirm no login is required.
8. Confirm mobile layout is usable.
9. Hammer the proxy: repeat fetches + a 4-way compare and confirm caching keeps it from rate-limiting; confirm sample-data backstop still works if the proxy is forced to fail.

Deliver:

- Production URL
- Smoke test summary
- Any limitations judges should know, if unavoidable

Prompt 11: Submission Assets Agent

Prepare ShipGotchi submission assets for Dollar Vibe Club.

Read:

- docs/06-judging-demo-submission-kit.md

Deliver:

1. Final project name.
2. Final tagline.
3. Final under-500-character submission description.
4. 10 tags.
5. Cover image concept or actual cover image.
6. Optional 60-90 second demo video script.
7. Three working features to list clearly.

Submission details:

Project name: ShipGotchi

Tagline: Paste your GitHub. Meet the creature your commits created.

Working features:

1. GitHub username hatch.
2. Activity-based pet evolution.
3. Builder DNA and Compare Mode.

Make the submission sound polished, launchable, and easy to judge.

Prompt 12: Final Judge Demo Rehearsal

Run a final judge demo rehearsal for ShipGotchi.

Use the production URL.

Demo path:

1. Open the app.
2. Click a sample username.
3. Wait for hatch.
4. Explain the pet mood, evolution, level, and Builder DNA.
5. Add another username to Compare Mode.
6. Show side-by-side difference.
7. Show share/result card.

Time target:

The core value should be obvious within 30 seconds.

Check:

- Is the first screen instantly understandable?
- Does the pet feel like the hero?
- Are the five MVP features visible?
- Does the app recover gracefully from bad input?
- Is mobile clean?
- Would a stream audience understand it without explanation?

Deliver:

A final go/no-go recommendation and a short list of only must-fix issues.

Prompt 13: Emergency Scope Cut

Use this prompt if time is running out.

We are close to deadline. Cut ShipGotchi down to the strongest shippable judging demo.

Protect these features:

1. GitHub hatch or sample-data hatch.
2. Pet mood/evolution visual.
3. Builder DNA card.
4. Mobile-friendly production URL.
5. No-login judging path.

Cut in this order:

1. Desk Mode.
2. Share card export/download.
3. Outfit overlays.
4. Advanced comparison awards.
5. Advanced language fetching.

Task:

Identify unstable or unfinished features, remove or hide them, and make the remaining path feel intentional and polished.

Acceptance:

A judge can still complete a delightful demo in under 60 seconds.

Prompt 14: Post-Sprint Vision Pitch

Use this for the submission, demo narration, or follow-up if ShipGotchi places.

Write a concise post-sprint vision pitch for ShipGotchi.

Positioning:

ShipGotchi starts as a no-login web toy that turns public GitHub activity into a living builder pet. After the sprint, it can become a persistent companion that lives beside your work and grows as you ship.

Future expansion ideas:

- Desktop/web companion mode
- Browser extension
- Team pets
- Streak celebrations
- Seasonal evolution forms
- Shareable launch cards
- DVC builder community leaderboard

Keep it concise, exciting, and realistic. Emphasize that the sprint MVP already proves the core loop: public activity → emotional pet → shareable identity.

Completion Checklist

Before submitting, confirm:

- The app is deployed to a public Vercel URL.
- GITHUB_TOKEN is set in Vercel env and the /api/github proxy returns live data in production.
- Caching confirmed: repeated fetches + a 4-way compare do not rate-limit.
- Sample-data backstop works even if the proxy is forced to fail.
- No login is required.
- Sample usernames work.
- At least one real GitHub username works.
- Invalid username state works.
- Pet state visibly changes across users.
- Builder DNA card is readable on mobile.
- Compare Mode works.
- Cover image is ready.
- Submission copy is under 500 characters.
- The production URL will stay online through judging.